



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS**

A Minor Project Report On
VisionProctor
AI-Based Online Exam Proctoring System

Submitted By:

Alex Shrestha (080BCT012)
Darpan Giri (080BCT024)
Mahesh Bhandari (080BCT045)

Submitted To:

Department of Electronics and Computer Engineering
Pulchowk Campus, Lalitpur

August 22, 2026

Acknowledgement

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, for providing us with the opportunity, resources, and academic environment to carry out this project. We are especially grateful to our project supervisor, Mr. Prakash Chandra Prasad, Assistant Professor, Department of Electronics and Computer Engineering, for his continuous guidance, valuable suggestions, and constructive feedback throughout the development of this project. His technical insights and encouragement have helped us refine our ideas and improve the overall quality of our work. We would also like to thank all the faculty members of the department for their valuable support and guidance during the course of this project. Their knowledge and suggestions have contributed significantly to our learning and to the successful completion of this work.

Finally, we are thankful to everyone who directly or indirectly supported us during the development of this project. Their encouragement and assistance have been valuable throughout this journey.

Alex Shrestha (080BCT012)

Darpan Giri (080BCT024)

Mahesh Bhandari (080BCT045)

Abstract

The rapid shift towards online learning and remote examinations has exposed the limitations of traditional human invigilation, which is costly, difficult to scale, and prone to inconsistency. This project, **VisionProctor**, proposes an AI-based online proctoring system that automates the detection of common exam malpractices in real time. The system uses a pretrained MediaPipe Face Mesh model (468 facial landmarks) to perform gaze estimation and head pose tracking, and a YOLOv8 object detection model, fine-tuned on a custom dataset, to detect mobile phones and multiple persons within the webcam frame. In addition, a browser-level monitoring module detects tab-switching and other suspicious window activity during the exam session. Each module independently monitors its respective signal and logs suspicious events in real time for administrator review. The system is designed as a lightweight, CPU-friendly, and modular alternative to expensive commercial proctoring solutions such as Proctorio and Respondus, aiming to reduce false positives through temporal smoothing of individual detection signals.

Keywords: Online Proctoring, Computer Vision, MediaPipe, YOLOv8, Object Detection, Gaze Tracking, Head Pose Estimation, Exam Integrity.

Contents

Acknowledgement	1
Abstract	2
List of Figures	6
List of Tables	7
List of Abbreviations	8
1 Introduction	9
1.1 Background	9
1.2 Problem Statement	9
1.3 Objectives	10
1.4 Scope	10
2 Literature Review	11
2.1 Related Work	11
2.2 Research Gap	12
2.3 Related Theory	13
2.3.1 Facial Landmark Detection (MediaPipe Face Mesh)	13
2.3.2 Head Pose Estimation	13
2.3.3 You Only Look Once (YOLO) Object Detection	13
2.3.4 Event Logging and Temporal Smoothing	14
2.3.5 Cross-Entropy Loss and Model Training	14
3 Methodology	15
3.1 Software Development Methodology	15
3.2 Feasibility Study	15
3.3 Requirement Analysis	16
3.3.1 Functional Requirements	16

3.3.2	Non-Functional Requirements	16
3.4	System Development	17
3.4.1	Gaze and Head Pose Tracking Module	17
3.4.2	Object and Multiple-Person Detection Module	17
3.4.3	Tab-Switching Detection Module	17
3.4.4	Event Logging Engine	17
4	Experimental Setup	18
4.1	Dataset	18
4.2	Dataset Preprocessing	18
4.3	Model Selection	19
4.4	Training Configuration (YOLOv8 Fine-Tuning)	19
4.5	Evaluation Metrics	19
4.6	Real-Time Performance Considerations	20
4.7	Testing Scenarios	20
5	System Design	21
5.1	Architecture Design	21
5.1.1	Client-Server Architecture	21
5.1.2	AI Processing Pipeline	23
5.1.3	MVC Design Pattern	23
5.2	Security Measures and Mechanisms	25
5.2.1	Authentication	25
5.2.2	Encryption	25
5.3	Use Case Diagram	25
5.4	Activity Diagram	27
6	Results	28
6.1	Object Detection Performance	28
6.2	Calibration: Gaze and Head Pose Tracking	30
6.3	Qualitative Results: Gaze and Head Pose Tracking	31
6.4	Qualitative Results: Object and Multiple-Person Detection	32
6.5	Qualitative Results: Tab Switch Detection	32
6.6	Qualitative Results: Person Absent	33
6.7	Event Log Summary	34

7	Conclusion	35
8	Limitations and Future Enhancements	36
8.1	Limitations	36
8.2	Future Enhancements	36
	References	38

List of Figures

3.1	Agile Development Methodology	15
5.1	Client-Server Architecture	21
5.2	VisionProctor System Architecture	22
5.3	MVC Diagram for VisionProctor	24
5.4	Use Case Diagram for VisionProctor	26
5.5	Activity Diagram for VisionProctor	27
6.1	Precision-Recall Curve for fine-tuned YOLOv8 model	28
6.2	Training and validation loss curves (box, classification, and DFL losses) and object detection performance metrics (precision, recall, mAP@50, and mAP@50-95) during YOLOv8 fine-tuning	29
6.3	Confusion Matrix for fine-tuned YOLOv8 model on the test set	29
6.4	Eye calibration	30
6.5	Head calibration	30
6.6	Sample frames with head-pose	31
6.7	Sample frames with gaze	31
6.8	Sample detection results: mobile phone	32
6.9	Live Tab Switch Detection	32
6.10	Tab Switch Detection snapshot	33
6.11	Person Absent Snapshot	33
6.12	Logged suspicious events over time for a sample exam session	34

List of Tables

2.1	Comparative summary of existing proctoring approaches	12
4.1	Dataset summary	18
6.1	YOLOv8 fine-tuning results	29

List of Abbreviations

AI	Artificial Intelligence
CNN	Convolutional Neural Network
YOLO	You Only Look Once
API	Application Programming Interface
JWT	JSON Web Token
REST	Representational State Transfer
COCO	Common Objects in Context
FPS	Frames Per Second
GPU	Graphics Processing Unit
CPU	Central Processing Unit
SDLC	Software Development Life Cycle
MVC	Model-View-Controller
UI	User Interface

Chapter 1

Introduction

1.1 Background

The post-pandemic era has witnessed a rapid and sustained adoption of online learning and remote examinations across academic institutions. While this shift has improved accessibility and flexibility, it has simultaneously introduced significant challenges in maintaining examination integrity. Traditional human invigilation, effective in physical classrooms, becomes costly and impractical to scale when examinations are conducted remotely for large cohorts of students.

Common integrity risks in remote examinations include the use of unauthorized devices such as mobile phones, consultation of external material, presence of unauthorized persons in the exam environment, and switching between browser tabs or applications to access unauthorized resources. Manually monitoring hundreds of live video feeds simultaneously is infeasible, creating a strong need for automated, AI-driven proctoring solutions capable of providing real-time, objective, and scalable monitoring.

1.2 Problem Statement

Existing online examination platforms largely lack integrated, real-time mechanisms to detect malpractice. Where automated proctoring tools do exist, they are often expensive, closed-source commercial products (e.g. Proctorio, Respondus) that are inaccessible to smaller institutions, or academic prototypes that detect only a narrow set of behaviours (e.g. gaze direction alone, or object detection alone). Furthermore, many existing systems suffer from high false-positive rates under varying lighting conditions and webcam quality, reducing their practical reliability.

There is a clear need for a lightweight, modular, and reasonably accurate system that can:

- Continuously monitor a candidate’s gaze and head orientation to detect off-screen

attention.

- Detect the presence of unauthorized objects (e.g. mobile phones) and unauthorized persons within the camera frame.
- Monitor browser-level activity such as tab switching.
- Log and flag these events with minimal false alarms.

1.3 Objectives

The primary objectives of this minor project are:

- To track a candidate's eye gaze and head pose in real time using a pretrained facial landmark detection model.
- To detect mobile phones and multiple persons in the exam video feed using a YOLOv8 object detection model fine-tuned on a custom dataset.
- To implement a browser tab-switching detection module to flag window or tab-focus changes during the exam.
- To design an event logging and alerting mechanism that flags suspicious behaviour detected by each module for administrator review.
- To build a scalable client-server system that supports real-time video streaming, exam management, and reporting of proctoring analytics.

1.4 Scope

The scope of this project covers the design and implementation of a real-time, browser-based AI proctoring system for single-candidate online examinations. It includes gaze and head pose tracking, object and person detection, and tab-switch monitoring, each independently logging and flagging suspicious events, along with a web-based front end for students and an administrative interface for exam creators. Face recognition-based identity verification is outside the current scope of the implemented system; the discussion below on identity risks pertains to the broader problem motivation rather than an implemented feature. Future extensions such as audio-based cheating detection and mouth-movement analysis are outlined as potential future work.

Chapter 2

Literature Review

2.1 Related Work

Automated proctoring has been explored both by commercial vendors and academic researchers. Commercial platforms such as Proctorio combine face detection, eye tracking, and browser-lockdown features to provide fully automated monitoring, but are often expensive and have been reported to struggle with reliable object detection and head-pose estimation under varied conditions. Respondus similarly combines webcam monitoring and browser lockdown to prevent access to unauthorized applications, but remains closed-source with limited object-detection capability. Academic work, such as the CNN-based proctoring approach of Chen and Alvarez, has reported good accuracy (approximately 91%) for detecting suspicious activity, but typically focuses on a single behavioural signal and does not integrate browser-level monitoring such as tab-switch detection.

Table 2.1: Comparative summary of existing proctoring approaches

Author / Approach	Method	Strengths	Limitations
Proctorio	Face detection, eye tracking, browser monitoring	Fully automated	High cost; fails with object detection & head pose tracking
Chen & Alvarez	CNN-based proctoring	Good accuracy ($\sim 91\%$)	Focuses mainly on suspicious activity; no object detection; no tab monitoring
Respondus	Webcam monitoring + face detection + browser lockdown	Prevents opening other applications via LockDown Browser	Closed source; limited object detection

2.2 Research Gap

A review of existing systems reveals a set of recurring limitations: most detect only one or two behavioural cues (either gaze/pose *or* objects, rarely both), suffer from a high rate of false positives under varying lighting conditions, and are expensive commercial services.

VisionProctor is positioned to address these gaps by combining:

- MediaPipe’s 468-point facial landmark model for real-time, CPU-efficient gaze and head pose estimation.
- A fine-tuned YOLOv8 model for detecting mobile phones and multiple persons.
- Independent monitoring of gaze/pose, object/person, and browser tab-switch events, each logged and flagged in real time.
- A lightweight, CPU-friendly design suitable for real-time deployment at lower cost than commercial alternatives.

The overall design goal is to reduce false positives in each detection module through temporal smoothing, rather than relying on a single instantaneous reading.

2.3 Related Theory

2.3.1 Facial Landmark Detection (MediaPipe Face Mesh)

MediaPipe Face Mesh is a pretrained, lightweight machine learning model that estimates 468 3D facial landmarks from a single RGB image in real time, without requiring specialized hardware such as depth sensors. In VisionProctor, this model is used purely for behavioural monitoring, not identity recognition: the geometric relationship between eye and iris landmarks is used to estimate gaze direction, while the relative position of landmarks such as the nose tip, eye corners, and chin is used to estimate head pose (yaw, pitch, and roll).

2.3.2 Head Pose Estimation

Head pose is commonly estimated by solving the Perspective-n-Point (PnP) problem, which relates a set of known 3D facial landmark positions to their corresponding 2D projections in the image plane:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K [R \mid t] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (2.1)$$

where K is the camera intrinsic matrix, R and t are the rotation and translation of the head relative to the camera, and (X, Y, Z) are 3D landmark coordinates. The resulting rotation matrix R is decomposed into yaw, pitch, and roll angles, which are thresholded to flag sustained off-screen attention.

2.3.3 You Only Look Once (YOLO) Object Detection

YOLO formulates object detection as a single regression problem, directly predicting bounding boxes and class probabilities from full images in one forward pass through the network, making it well suited to real-time applications. YOLOv8 is the version used in this project. The base model is pretrained on the COCO dataset, capable of detecting common classes such as *person* and *cell phone*, and is further fine-tuned on a custom-labelled dataset to improve detection accuracy specifically for the exam proctoring use case.

2.3.4 Event Logging and Temporal Smoothing

Each detection module (gaze/head pose, object/person, tab switch) independently raises an event when its respective condition is met (e.g. sustained gaze deviation, a detected mobile phone, more than one person in frame, or a tab-switch action). To avoid raising false alarms from single-frame noise or momentary detections, each module applies temporal smoothing before logging an event – for instance, requiring a behaviour to persist for a minimum duration, or smoothing the raw per-frame signal (e.g. via an exponential moving average) before comparing it against a threshold. Flagged events are timestamped, logged, and made available to the administrator for review.

2.3.5 Cross-Entropy Loss and Model Training

Fine-tuning of the YOLOv8 detection head uses a composite objective combining classification loss, localization (bounding box) loss, and objectness loss. The classification component follows the standard cross-entropy formulation:

$$L_{cls} = - \sum_i y_i \log(p_i) \quad (2.2)$$

where p_i is the predicted class probability and y_i is the ground-truth label.

Chapter 3

Methodology

3.1 Software Development Methodology

This project follows the **Agile** software development methodology. Agile allows for iterative development and testing of individual modules (gaze tracking, object detection, tab-switch monitoring, fusion engine) and supports incorporation of feedback after each testing cycle. This is particularly useful given the exploratory nature of tuning detection thresholds and fusion weights.

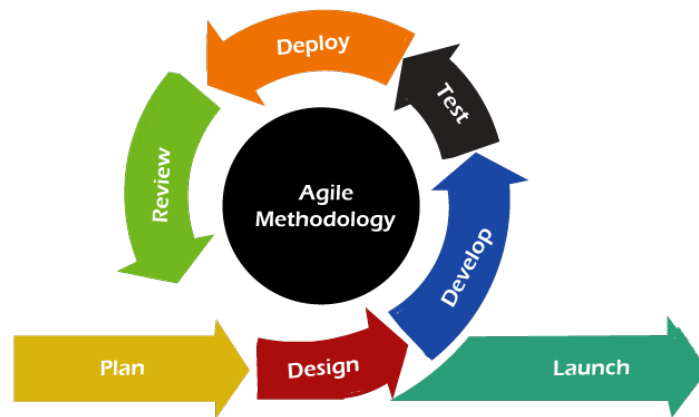


Figure 3.1: Agile Development Methodology

3.2 Feasibility Study

The project is technically feasible given the team’s familiarity with Python and the availability of pretrained MediaPipe and YOLOv8 models, which significantly reduce the need for training models from scratch. Financially, the project can be developed and tested using free-tier cloud GPU resources (Google Colab, Kaggle) for fine-tuning YOLOv8, along with personal laptops for development, keeping the project within a modest budget. The client-server architecture, built with Next.js and FastAPI, uses widely documented,

open-source frameworks, further supporting feasibility within the project timeline.

3.3 Requirement Analysis

3.3.1 Functional Requirements

- **Registration and Login:** Students and administrators can register and authenticate into the system.
- **Exam Creation and Management:** Admins can create, configure, and manage exams.
- **Live Video Monitoring:** The system captures the student's webcam feed during the exam session.
- **Gaze and Head Pose Tracking:** The system continuously estimates gaze direction and head orientation using MediaPipe.
- **Object and Person Detection:** The system detects mobile phones and counts the number of persons visible using the fine-tuned YOLOv8 model.
- **Tab-Switch Detection:** The system detects and logs browser tab or window focus changes.
- **Event Logging and Alerts:** The system logs flagged events from each detection module and raises an alert whenever a module's detection threshold is exceeded.
- **Reporting:** Admins can view logs, flagged events, and generate exam reports.

3.3.2 Non-Functional Requirements

- **Performance:** The system should process video frames with minimal latency to support real-time monitoring.
- **Scalability:** The system should support multiple concurrent exam sessions.
- **Reliability:** The system should degrade gracefully under poor network or lighting conditions.
- **Security:** Communication between client and server must be encrypted; access must be authenticated via JWT.
- **Usability:** The interface should be intuitive for both students and administrators.
- **Maintainability:** The modular architecture should allow independent updates to the detection models without affecting other components.

3.4 System Development

3.4.1 Gaze and Head Pose Tracking Module

This module uses the pretrained MediaPipe Face Mesh model to extract 468 facial landmarks per frame in real time. Eye landmark geometry is used to estimate gaze direction (on-screen vs. off-screen), while a subset of stable facial landmarks is used to solve the head pose estimation problem, yielding yaw, pitch, and roll angles. No fine-tuning was required for this module since the pretrained model generalizes well across poses and lighting conditions.

3.4.2 Object and Multiple-Person Detection Module

Unlike the gaze/pose module, the object detection module uses a **YOLOv8 model fine-tuned on a custom dataset** curated specifically for the exam-proctoring context (i.e. images/frames containing mobile phones, multiple persons, and typical exam-desk scenes). Fine-tuning on domain-specific data improves detection accuracy and reduces false positives relative to using the base COCO-pretrained weights alone.

3.4.3 Tab-Switching Detection Module

This module runs on the client side within the browser and listens for `visibilitychange` and window `blur/focus` events to detect when the candidate switches away from the exam tab or window. Each detected event is timestamped and sent to the backend as a suspicious-activity log entry.

3.4.4 Event Logging Engine

The event logging engine receives flagged events from all three detection modules (gaze/pose, object/person, tab-switch), as described in Section 2.2.4. Each event is timestamped, tagged with its source module, and stored along with a snapshot of the corresponding frame for administrator review.

Chapter 4

Experimental Setup

4.1 Dataset

A custom dataset was curated and annotated for fine-tuning the YOLOv8 object detection model, containing labelled instances of mobile phones and multiple persons in typical exam-desk webcam scenarios. The pretrained MediaPipe Face Mesh model was used as-is (without additional fine-tuning) for gaze and head-pose estimation.

Dataset link: <https://universe.roboflow.com/project-2morrow-software-limited/online-proctoring-system-x27ou>

Table 4.1: Dataset summary

Class	Number of Instances	Split (Train/Val/Test)
Person	13,334	Roboflow default split
Cell Phone	3,437	Roboflow default split
Book	1,935	Roboflow default split
TV	1,103	Roboflow default split
Laptop	702	Roboflow default split
Headphone	673	Roboflow default split

Total images: 15,489 (640×640, auto-oriented). Only the *person* and *cell phone* classes were used for this project’s multiple-person and mobile-detection modules.

4.2 Dataset Preprocessing

Images were resized and normalized to match the YOLOv8 input resolution. Standard data augmentation techniques such as random horizontal flipping, scaling, mosaic augmentation, and brightness/contrast jittering were applied to improve generalization under varying lighting conditions.

4.3 Model Selection

- **Gaze and Head Pose:** Pretrained MediaPipe Face Mesh (468 landmarks) – selected for its real-time, CPU-efficient performance and robustness across head poses, requiring no additional training.
- **Object / Person Detection:** YOLOv8, fine-tuned on the custom dataset – selected for its balance of speed and accuracy, making it suitable for real-time inference on modest hardware.

4.4 Training Configuration (YOLOv8 Fine-Tuning)

- **Base Model:** YOLOv8s (pretrained on COCO) – the small variant was chosen over the nano variant for improved accuracy while remaining fast enough for near real-time inference.
- **Framework:** Ultralytics
- **Optimizer:** Auto (Ultralytics default optimizer selection between SGD and AdamW)
- **Learning Rate:** Default Ultralytics initial learning rate (`lr = 0.01`)
- **Epochs:** 100
- **Batch Size:** 32
- **Image Size:** 640×640
- **Dataloader Workers:** 4
- **Early Stopping Patience:** 20 epochs without improvement
- **Dataset Caching:** Enabled (`cache=True`) for faster training
- **Augmentations:** Mosaic, horizontal flip, HSV jitter (Ultralytics default augmentation pipeline)
- **Hardware:** Google Colab, NVIDIA Tesla T4 GPU

4.5 Evaluation Metrics

The performance of the fine-tuned YOLOv8 model is evaluated using standard object detection metrics:

- **Precision and Recall**

- Mean Average Precision (mAP@0.5 and mAP@0.5:0.95)
- F1 Score
- Confusion Matrix

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (4.1)$$

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.2)$$

4.6 Real-Time Performance Considerations

Since the system is intended for live proctoring, inference latency and frame-rate throughput are critical. Techniques such as model quantization and adjustable frame sampling are explored to keep CPU usage low while maintaining acceptable detection responsiveness.

4.7 Testing Scenarios

The system was tested under varying lighting conditions, webcam qualities, and scenarios including: single candidate with no malpractice, candidate holding a mobile phone, a second person entering the frame, and candidate switching browser tabs during the exam.

Chapter 5

System Design

5.1 Architecture Design

5.1.1 Client-Server Architecture

VisionProctor follows a client-server architecture. The client, built with Next.js, captures the candidate's webcam feed and streams it to the backend over an encrypted connection (HTTPS/WebRTC). The FastAPI backend hosts the AI processing pipeline, which performs frame preprocessing, gaze/head-pose estimation, and object and person detection, before returning results and alerts to the client and storing session data in a PostgreSQL database.

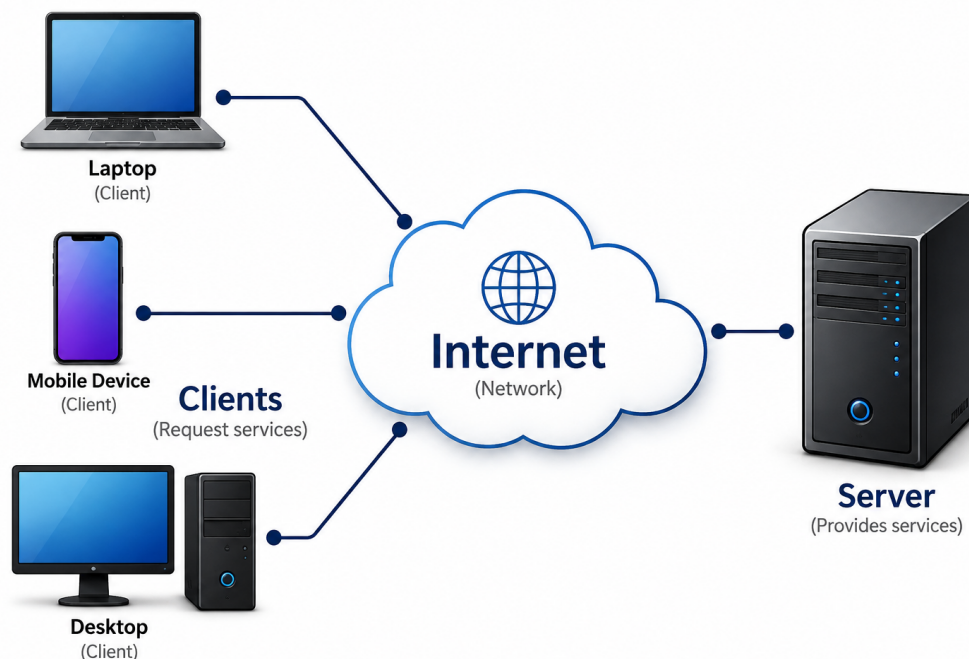


Figure 5.1: Client-Server Architecture

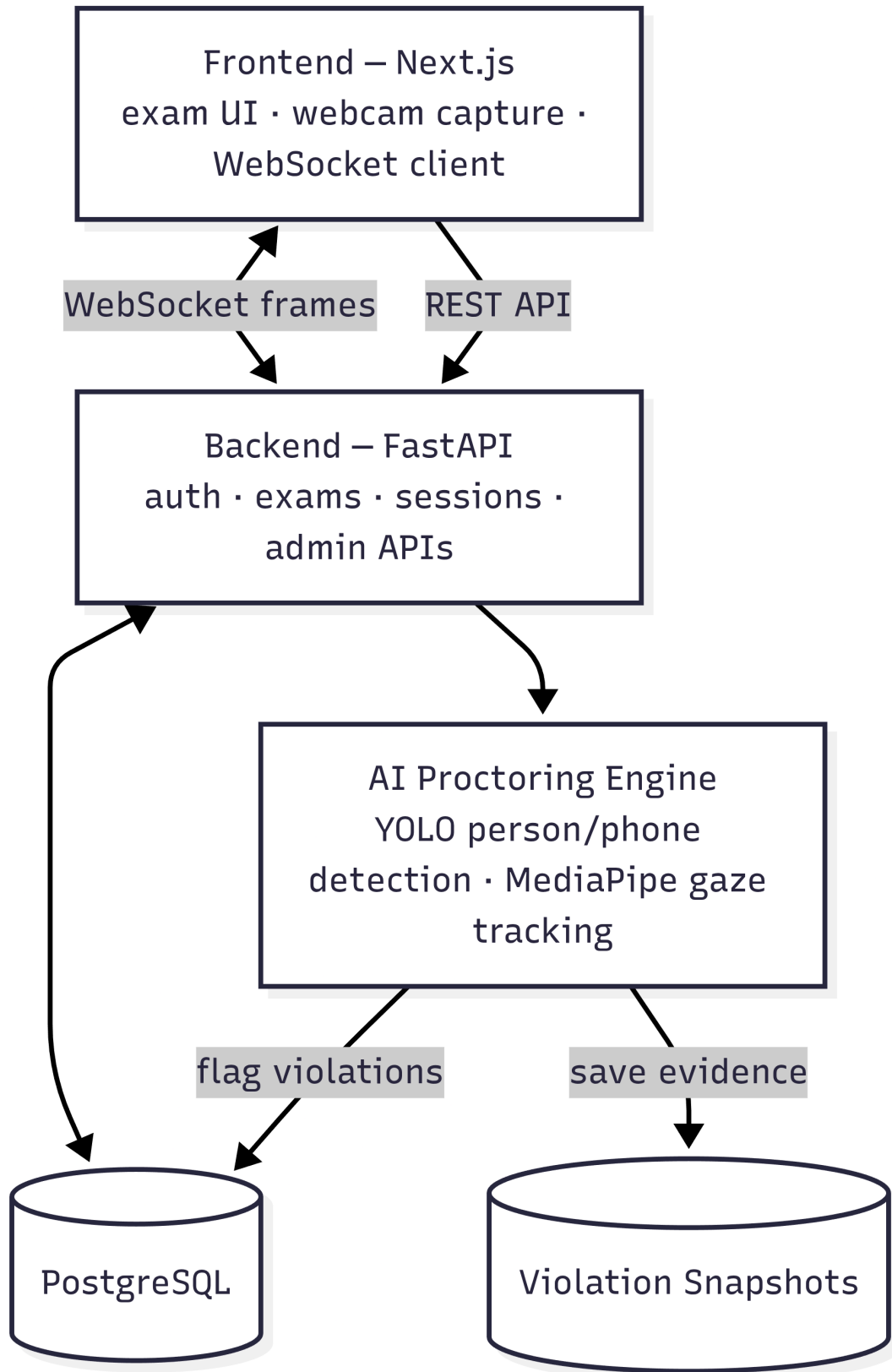


Figure 5.2: VisionProctor System Architecture

5.1.2 AI Processing Pipeline

The AI processing pipeline within the backend consists of the following stages:

1. **Image Processing:** Frame capture, resizing, normalization, and denoising using OpenCV.
2. **Gaze / Head Pose Estimation:** MediaPipe Face Mesh landmark extraction, followed by geometric computation of gaze direction and head pose angles.
3. **Object / Person Detection:** Fine-tuned YOLOv8 inference to detect mobile phones and count persons in frame.
4. **Event Logging Engine:** Receives flagged events from each detection module, timestamps and logs them, and triggers alerts when a module's detection threshold is exceeded.

5.1.3 MVC Design Pattern

The backend follows the Model-View-Controller (MVC) pattern:

- **Model:** Encapsulates the trained detection models, data preprocessing logic, and database interactions.
- **View:** The web-based user interface presenting live proctoring feedback, exam content, and results to students and administrators.
- **Controller:** Handles incoming requests (e.g. video frames, tab-switch events), invokes the appropriate model logic, and updates the view with results.

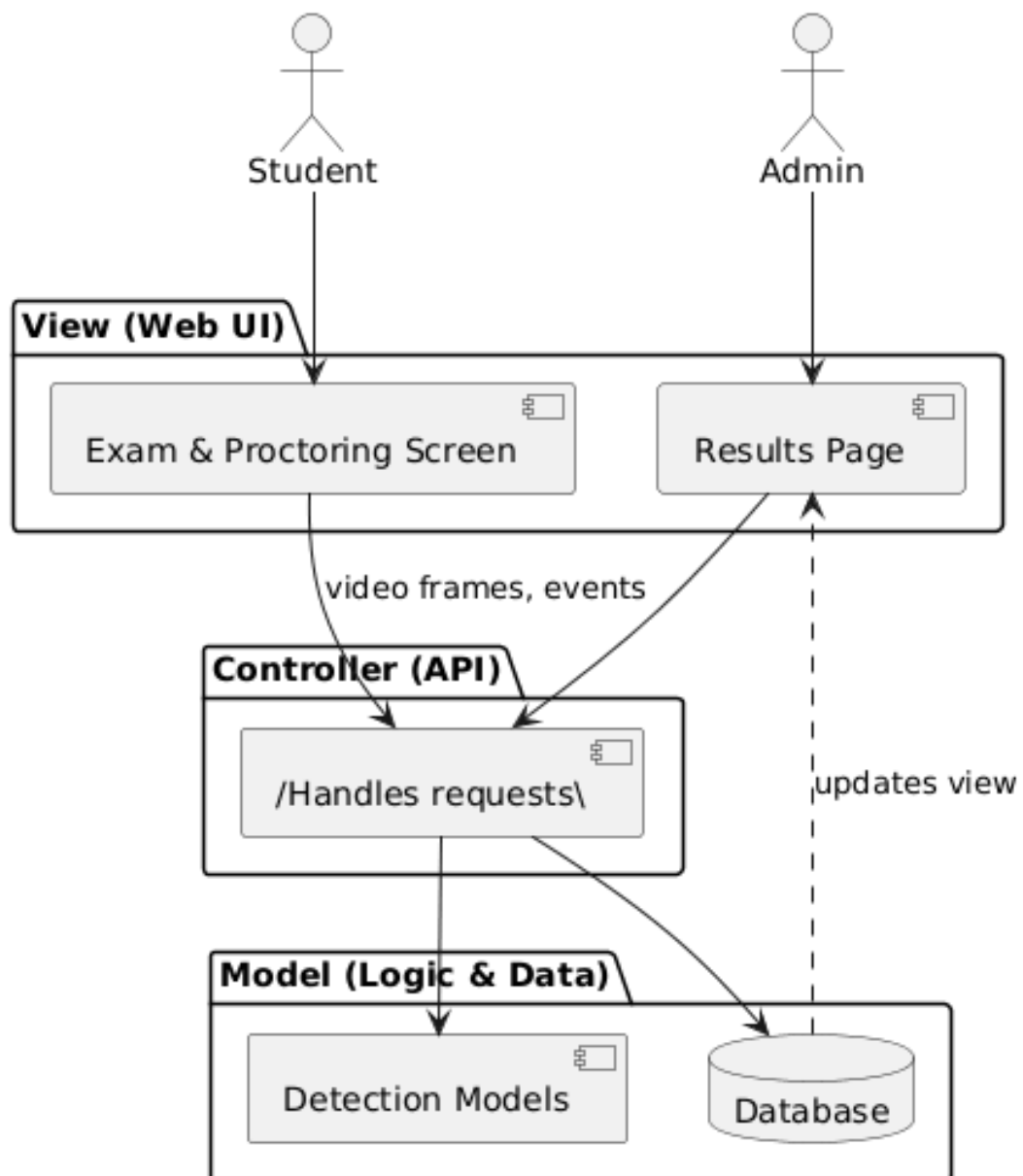


Figure 5.3: MVC Diagram for VisionProctor

5.2 Security Measures and Mechanisms

5.2.1 Authentication

The system uses JSON Web Tokens (JWT) for authentication. Upon successful login, the backend issues a JWT which the client includes in the **Authorization** header of subsequent requests using the Bearer scheme.

5.2.2 Encryption

All communication between client and server occurs over HTTPS. Sensitive data such as passwords are hashed using a secure algorithm (bcrypt) before storage.

5.3 Use Case Diagram

The primary actors of the system are the **Student** and the **Admin**. Students can register/login, undergo exam start procedures (which internally include head pose estimation, eye gaze tracking, browser tab-switch detection, object detection, and multiple-person detection), and submit exams. Admins can create exams, manage users, and view generated reports.

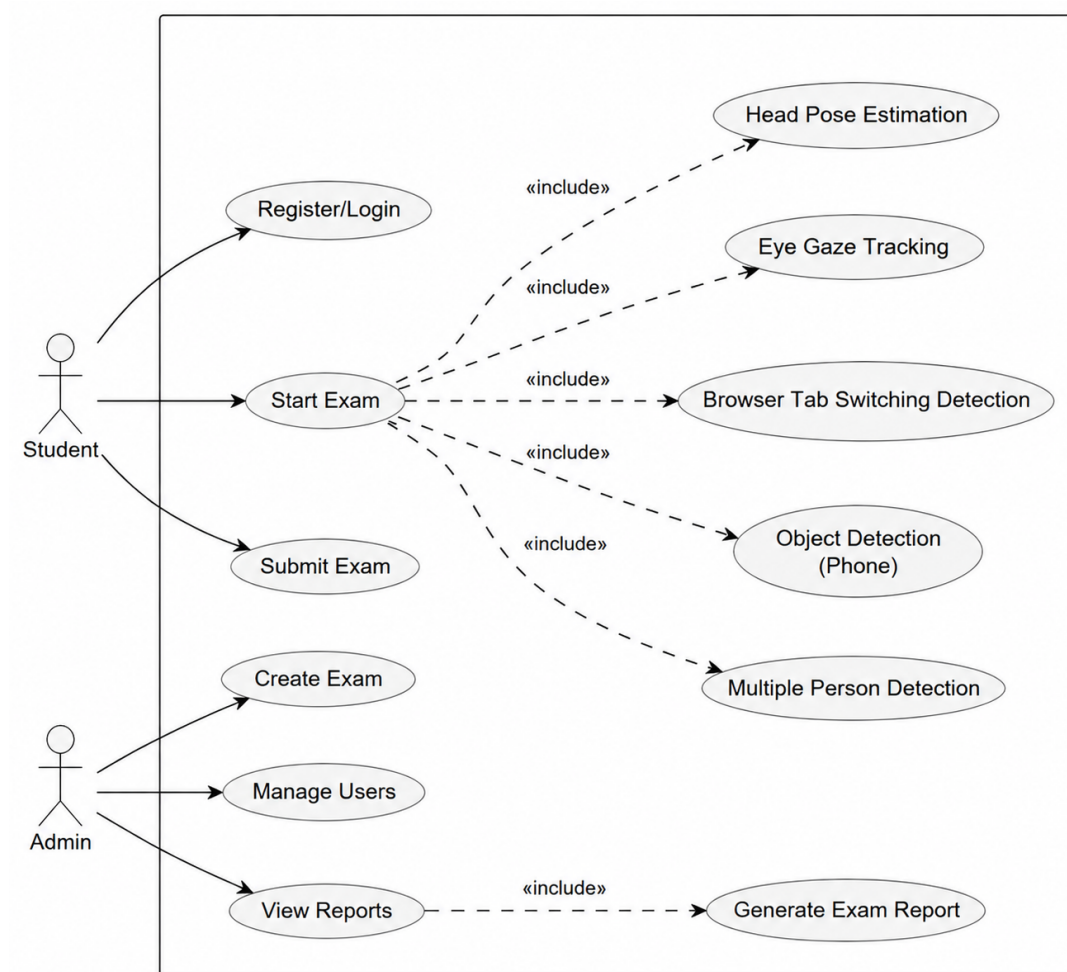


Figure 5.4: Use Case Diagram for VisionProctor

5.4 Activity Diagram

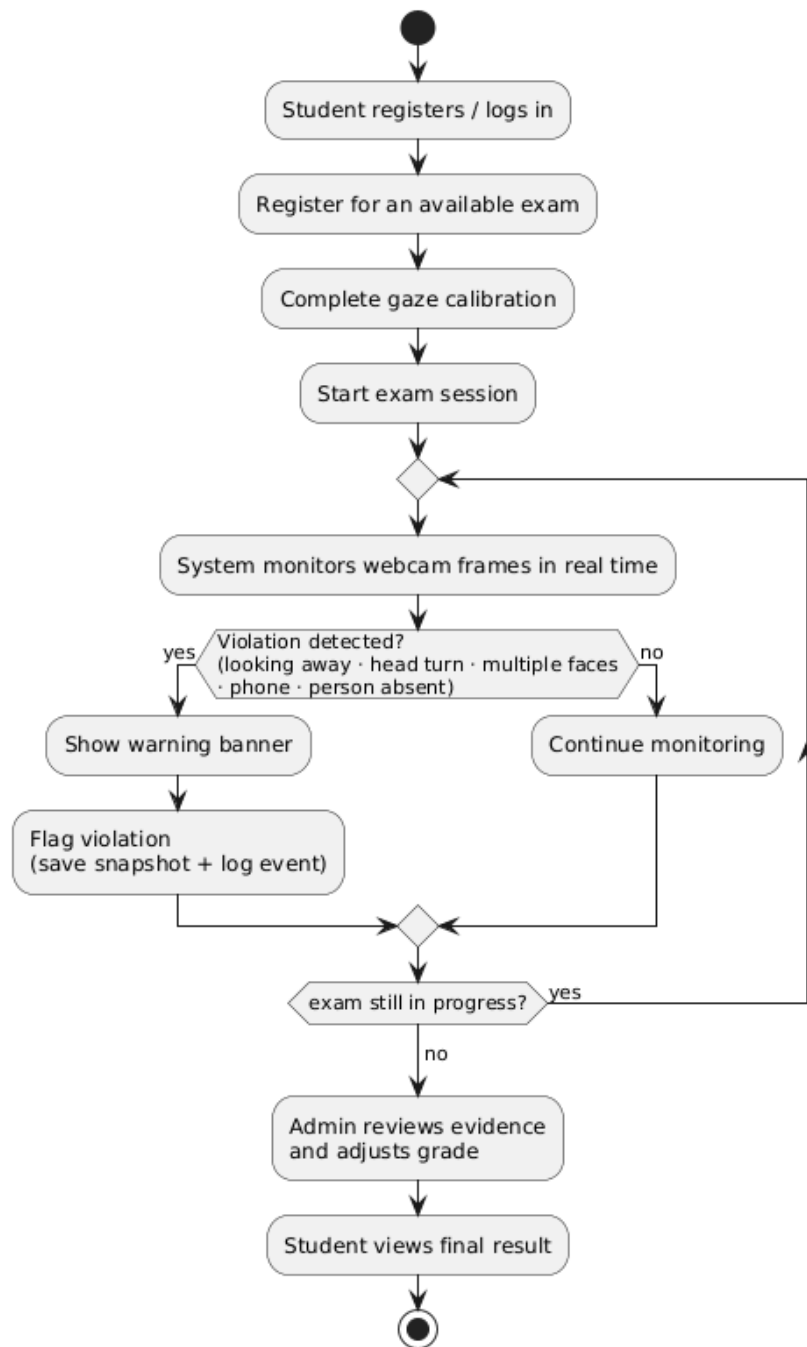


Figure 5.5: Activity Diagram for VisionProctor

Chapter 6

Results

This chapter presents the evaluation results of the fine-tuned YOLOv8 object/person detection model and qualitative observations of the integrated gaze, head pose, and tab-switch detection modules.

6.1 Object Detection Performance

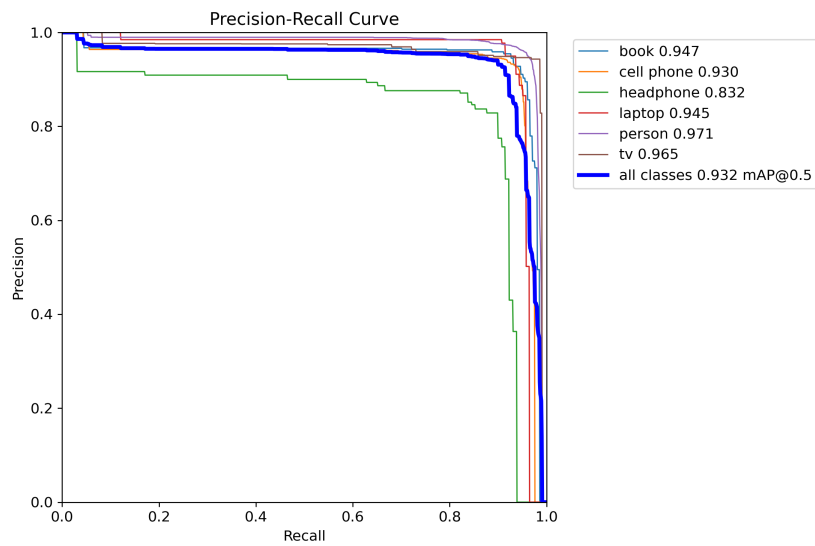


Figure 6.1: Precision-Recall Curve for fine-tuned YOLOv8 model

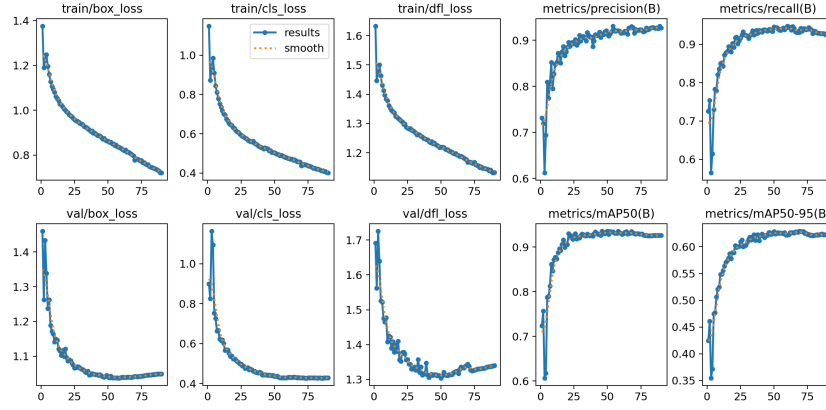


Figure 6.2: Training and validation loss curves (box, classification, and DFL losses) and object detection performance metrics (precision, recall, mAP@50, and mAP@50–95) during YOLOv8 fine-tuning

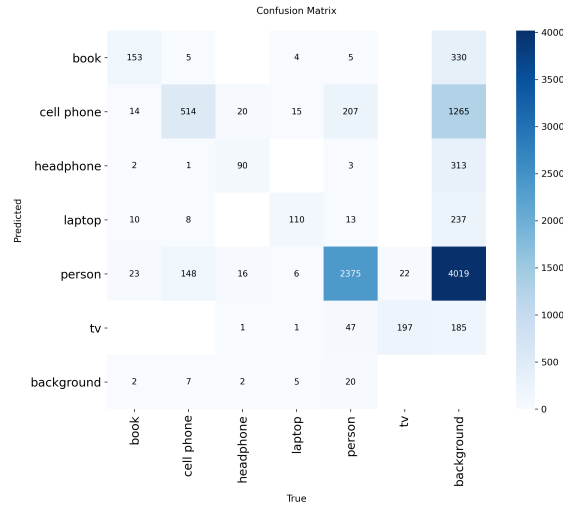


Figure 6.3: Confusion Matrix for fine-tuned YOLOv8 model on the test set

Table 6.1: YOLOv8 fine-tuning results

Metric	Value
Precision	0.92719
Recall	0.92608
mAP@0.5	0.9262
mAP@0.5:0.95	0.62223
F1 Score	0.92663

6.2 Calibration: Gaze and Head Pose Tracking

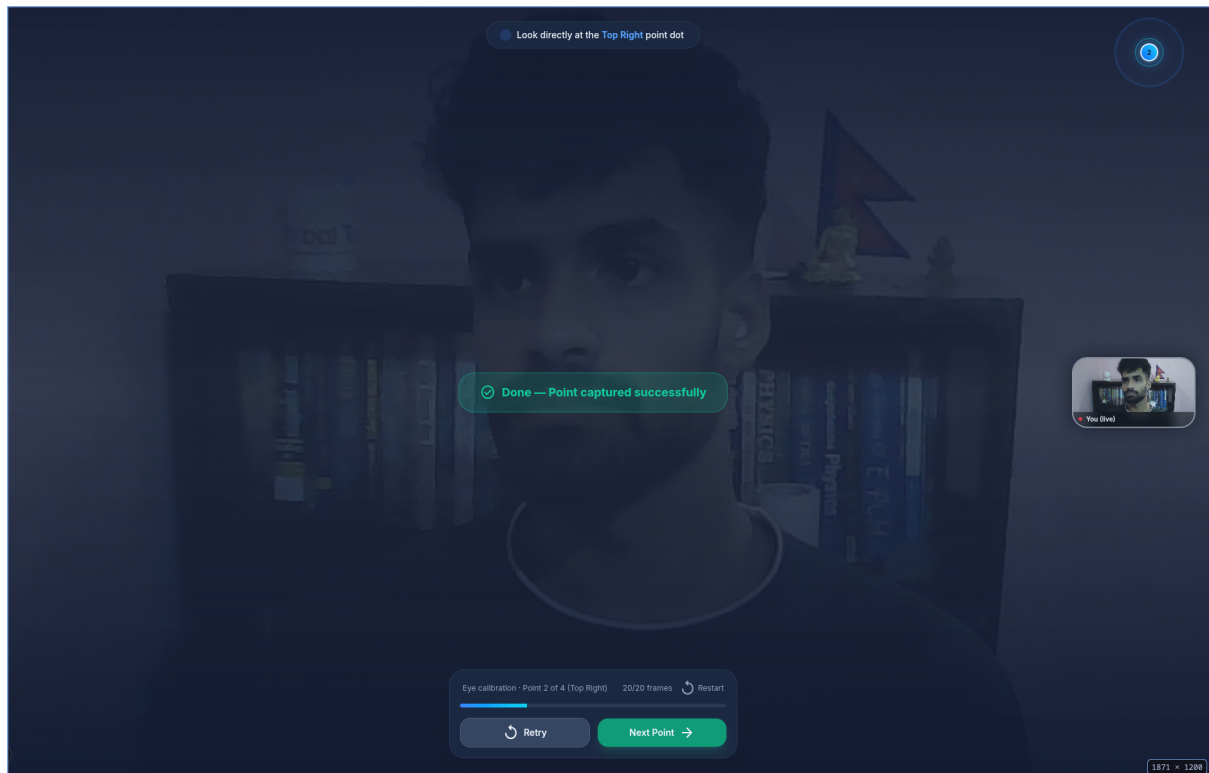


Figure 6.4: Eye calibration

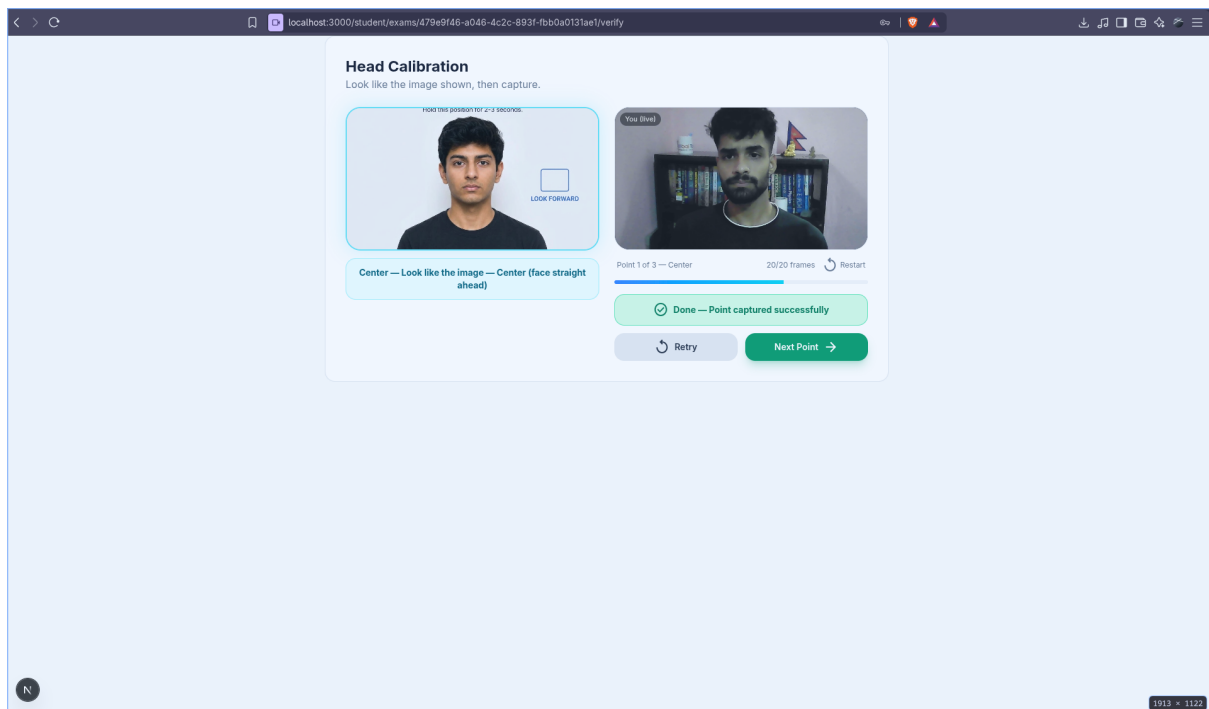


Figure 6.5: Head calibration

6.3 Qualitative Results: Gaze and Head Pose Tracking

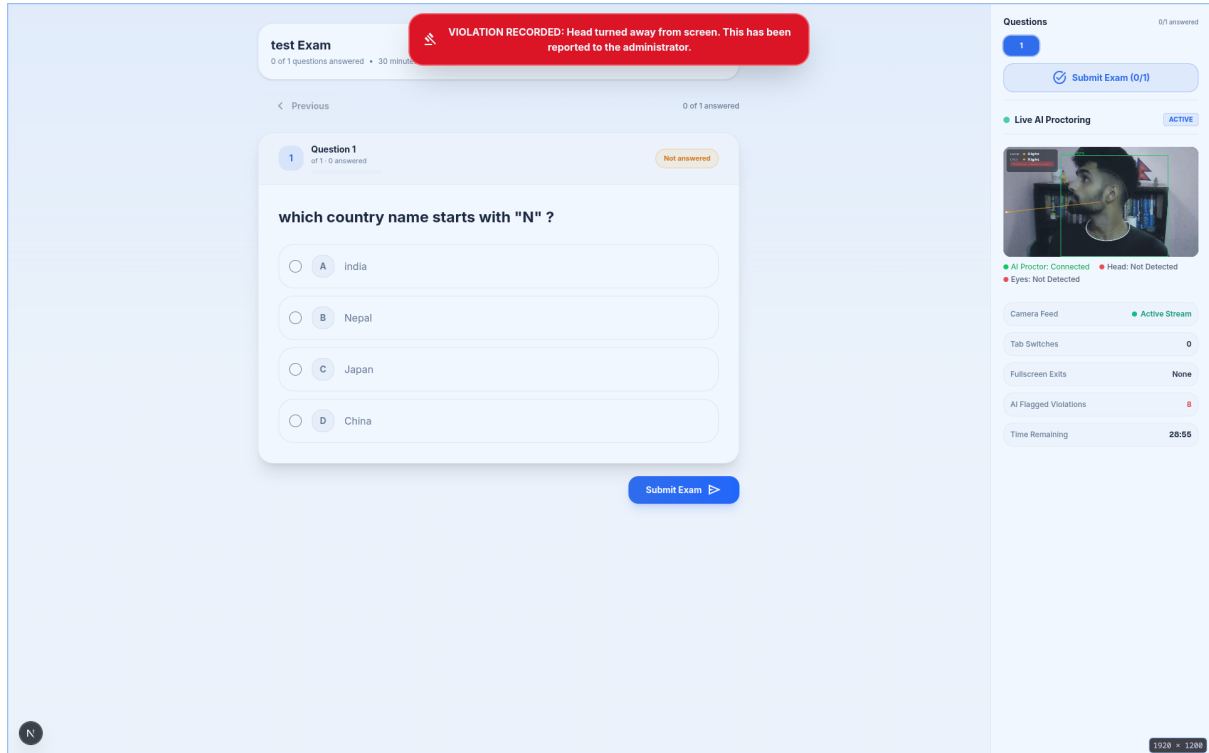


Figure 6.6: Sample frames with head-pose

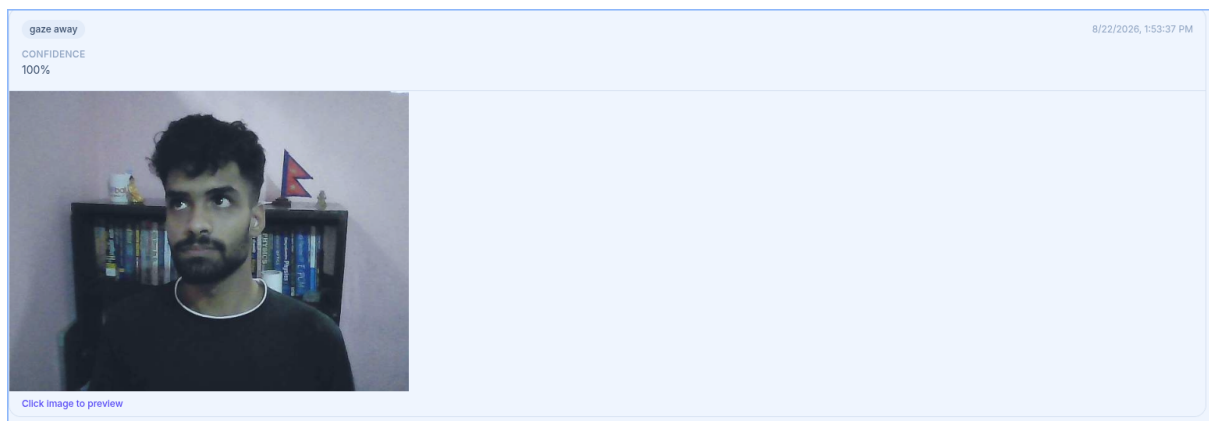


Figure 6.7: Sample frames with gaze

6.4 Qualitative Results: Object and Multiple-Person Detection

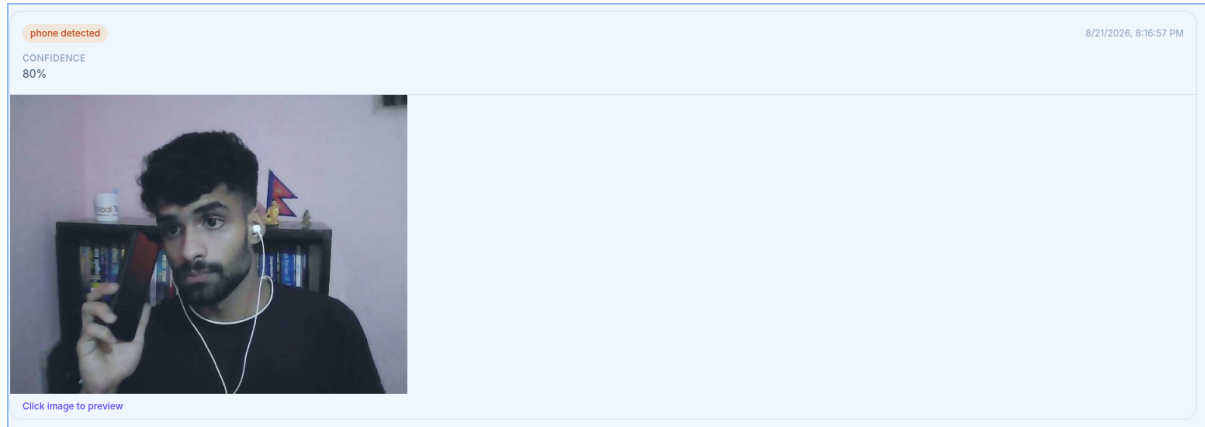


Figure 6.8: Sample detection results: mobile phone

6.5 Qualitative Results: Tab Switch Detection

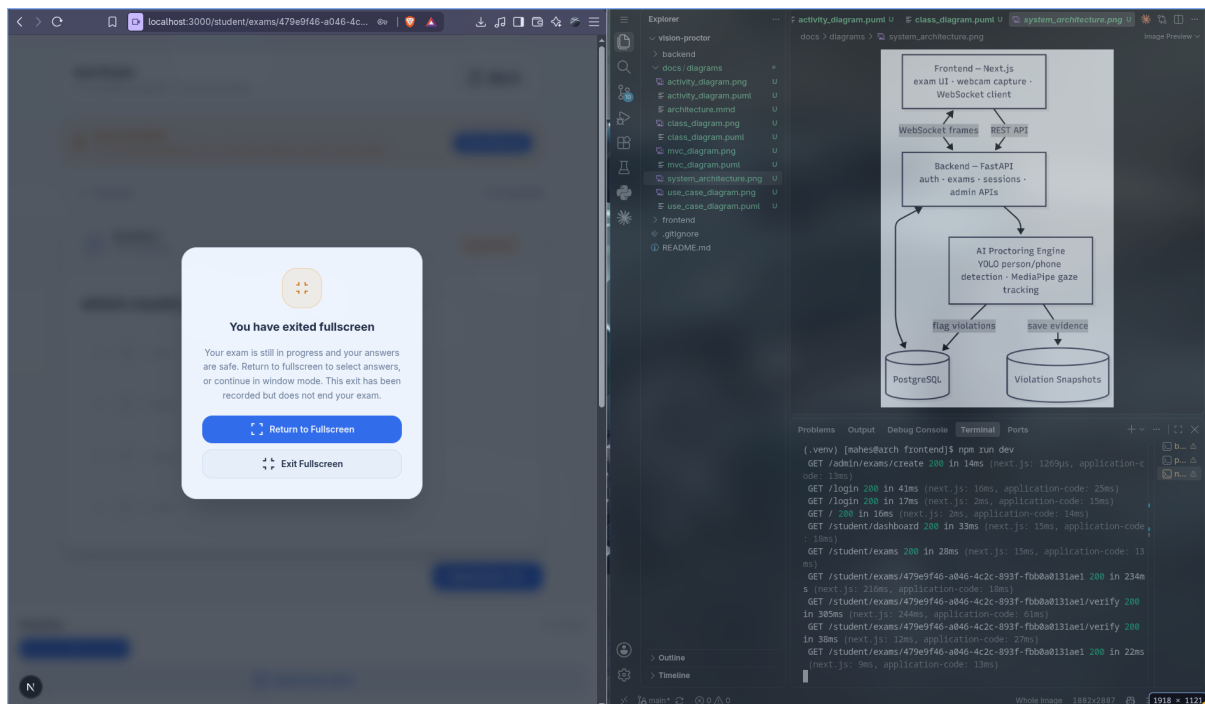


Figure 6.9: Live Tab Switch Detection

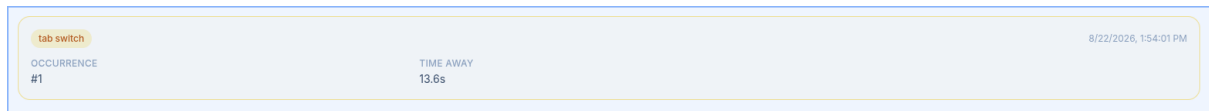


Figure 6.10: Tab Switch Detection snapshot

6.6 Qualitative Results: Person Absent

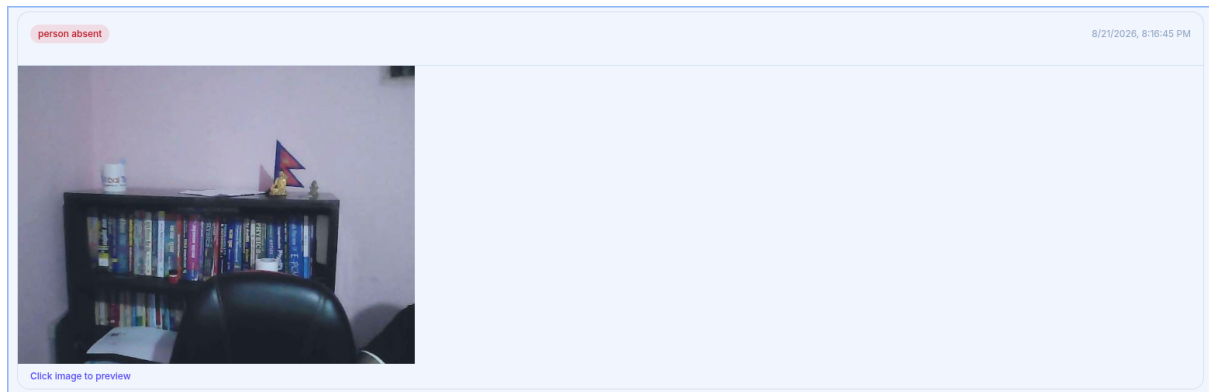


Figure 6.11: Person Absent Snapshot

6.7 Event Log Summary

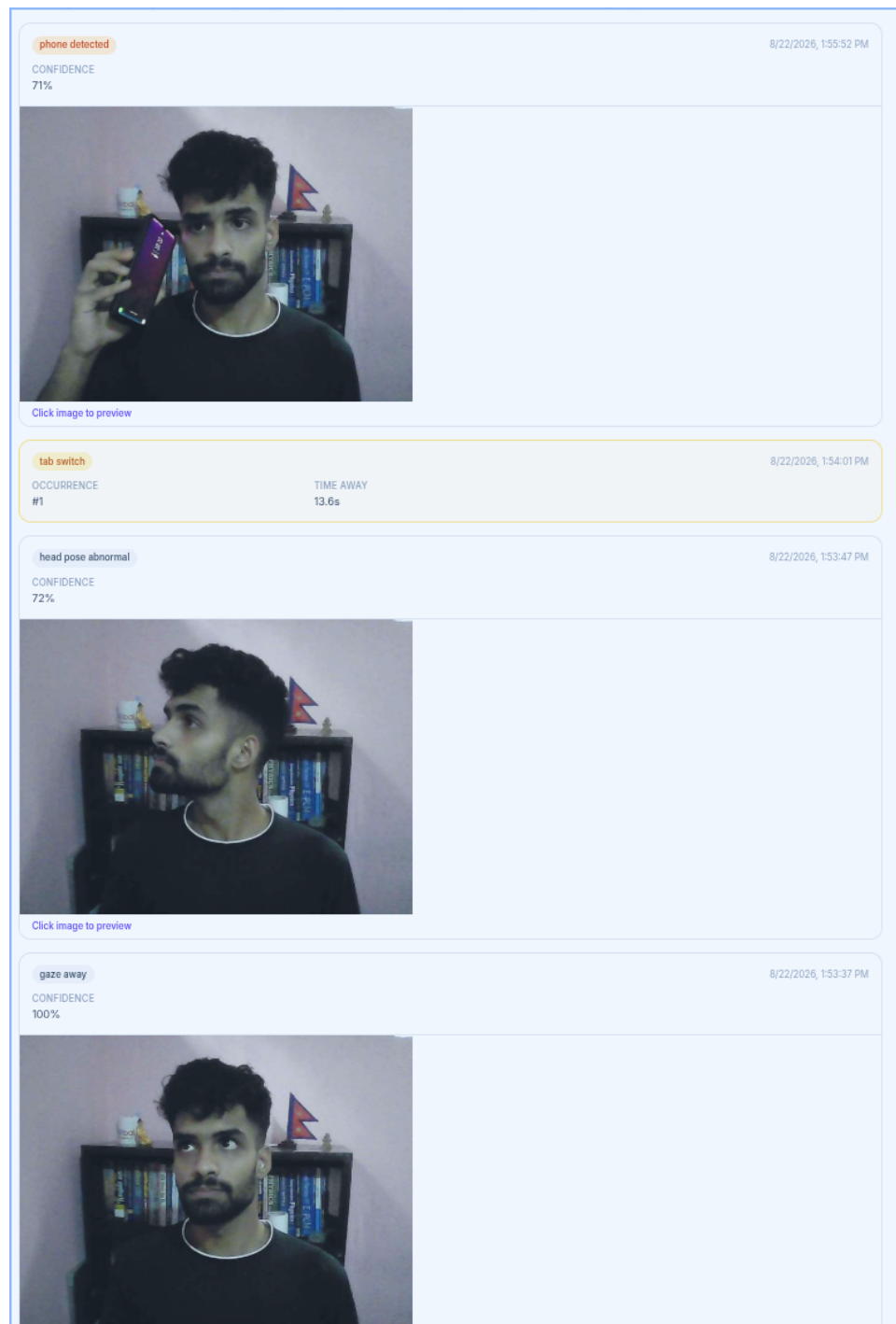


Figure 6.12: Logged suspicious events over time for a sample exam session

Chapter 7

Conclusion

This proposal presents **VisionProctor**, an AI-based online proctoring system designed to address the growing need for scalable, automated exam-integrity monitoring. By combining a pretrained MediaPipe Face Mesh model for real-time gaze and head-pose tracking with a YOLOv8 object detection model fine-tuned specifically for detecting mobile phones and multiple persons, along with a browser-level tab-switching detection module, the system independently monitors and logs suspicious behaviour from each of these signals. This modular approach, combined with per-module temporal smoothing, is intended to reduce the false-positive rates commonly observed in single-signal proctoring systems, while remaining lightweight and CPU-friendly compared to expensive commercial alternatives such as Proctorio and Respondus.

The proposed client-server architecture, built on Next.js and FastAPI with a PostgreSQL backend, provides a scalable foundation for real-time video streaming, exam management, and administrative reporting. While the current implementation focuses on gaze/pose tracking, object/person detection, and tab-switch monitoring, the modular design allows for future extension with additional behavioural signals such as audio-based cheating detection.

Chapter 8

Limitations and Future Enhancements

8.1 Limitations

1. **Lighting Sensitivity:** Gaze and head-pose estimation accuracy can degrade under poor or uneven lighting conditions.
2. **Occlusion Handling:** Partial occlusion of the face or the monitored object (e.g. a partially hidden phone) can reduce detection reliability.
3. **Computational Constraints:** Running both the landmark model and the object detector in real time can be demanding on lower-end CPUs without further optimization.
4. **False Positives from Ambiguous Behaviour:** Natural head movements or momentarily looking away from the screen may still trigger false alarms despite temporal smoothing.
5. **No Identity Verification:** The current system does not verify candidate identity; it focuses solely on behavioural monitoring during the exam session.
6. **Single-Camera Viewpoint Dependency:** Detection accuracy may vary depending on webcam placement and angle.

8.2 Future Enhancements

1. **Audio-Based Cheating Detection:** Incorporate audio analysis to detect conversations, whispering, or other suspicious sounds during the exam.

2. **Mouth Movement Detection:** Use facial landmarks to monitor lip movement and detect continuous speaking, which may indicate the candidate is talking to someone off-screen.
3. **Model Optimization:** Further fine-tune and quantize the YOLOv8 model on an expanded, more diverse dataset to improve accuracy and reduce inference latency.
4. **Edge Deployment:** Optimize the pipeline for on-device (client-side) inference to reduce server load and improve data privacy.
5. **Expanded Dataset:** Collect and annotate a larger and more diverse dataset covering different lighting conditions, camera qualities, and demographics.
6. **Adaptive Thresholding:** Explore adaptive, per-candidate detection thresholds for each module to further reduce false positives.
7. **Unified Suspicion Scoring:** As a future extension, explore combining the independently logged events from each module into a single weighted suspicion score to simplify administrator review.

References

1. Lugaresi, C., et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. *arXiv:1906.08172*.
2. Jocher, G., et al. (2023). Ultralytics YOLOv8. *GitHub Repository*.
3. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *CVPR*.
4. Lin, T.-Y., et al. (2014). Microsoft COCO: Common Objects in Context. *ECCV*.
5. Proctorio. *Official Product Documentation*.
6. Respondus. *LockDown Browser and Monitor Documentation*.
7. Chen, X., & Alvarez, R. (Year). CNN-Based Online Exam Proctoring. *PLACEHOLDER - Journal/Conference details*.